DYNAMIC PROGRAMMING MASTERCLASS

The Universal SB → RBR → MTS Framework

🧠 THE UNIVERSAL DP PARADIGM: SB → RBR → MTS

Never jump straight to a DP array! Every single DP problem from Fibonacci & House Robber to Knapsack, LCS, and Tree DP is solved in this exact 3-phase order:

🟦 SB — State & Brainstorm

S — State: What minimum variables uniquely identify subproblem `dp[i]` or `dp[i][j]`?

B — Brainstorm Choices: *"If I am at this state, what choices do I have?"* (Take / Skip, 1 step / 2 steps)

🟧 RBR — Recurrence, Base, Result

R — Recurrence: Convert choices to equation e.g. `dp[i] = max(take, skip)`

B — Base Case: Where does recursion stop? (`dp[0] = 1`)

R — Result: What answer to return? (`dp[N]` or `dp[M][N]`)

🟩 MTS — Memo, Tab, Space

M — Memoization: Cache recursive calls Top-Down

T — Tabulation: Convert recursion to loops Bottom-Up

S — Space Optimization: Compress array `dp[i]` to `prev1, prev2` variables!

💡 GOLDEN RULE: THE COMPLETE SOLUTION FLOW

```
Brute Force Decision Tree
  ↓
Recursive Function
  ↓
SB (Define State & Choices)
  ↓
RBR (Recurrence + Base Case + Result)
  ↓
MTS (Memoization → Tabulation → Space Optimizat
ion O(1))
```

Aha Moment: DP is simply *Optimized Recursion*! Tabulation is Memoization turned upside-down!

⚡ CLUES THAT SCREAM DYNAMIC PROGRAMMING

1. *"Find max/min cost or total number of ways to reach X"* \rightarrow 1D Linear DP
2. *"Target sum with reusable vs 1-time items"* \rightarrow Unbounded vs 0/1 Knapsack DP
3. *"Longest common subsequence / Edit distance of 2 strings"* \rightarrow 2D String DP

🎬 STEP-BY-STEP EVOLUTION OF CLIMBING STAIRS (LC 70)

Phase 1: SB — State & Brainstorm

- **State:** `dp[i]` = total ways to reach step `i`.
- **Choices at step `i`:** Arrive from step `i-1` (climb 1 step) OR step `i-2` (climb 2 steps).

Phase 2: RBR — Recurrence, Base, Result

- **Recurrence:** `dp[i] = dp[i-1] + dp[i-2]`
- **Base Case:** `dp[1] = 1`, `dp[2] = 2`
- **Result:** `return dp[n]`

Phase 3: MTS — Memoization ➔ Tabulation ➔ Space Opt

We transform naive recursion O(2^N) into memoization O(N), then tabulation O(N), and finally compressed O(1) space!

PATTERN 1 HOUSE ROBBER (1D Include / Exclude Choice) e.g. House Robber (LC 198)

SB → RBR BREAKDOWN

State: `dp[i]` = max money robbed up to house `i`.
Brainstorm Choices: Rob house `i` (`nums[i] + dp[i-2]`) OR Skip house `i` (`dp[i-1]`).
Recurrence: `dp[i] = max(nums[i] + dp[i-2], dp[i-1])`.
Base: `dp[0] = nums[0]`, `dp[1] = max(nums[0], nums[1])`.

TEMPLATE — LC 198 (SPACE OPTIMIZED O(1))

```
public int rob(int[] nums) {
  if (nums == null || nums.length == 0) return 0;
  int prev2 = 0, prev1 = 0;
  for (int num : nums) {
    int tmp = prev1;
    prev1 = Math.max(prev2 + num, prev1); // Rob or Skip!
    prev2 = tmp;
  }
  return prev1;
}
```

PATTERN 2 COIN CHANGE (Unbounded Knapsack Minimum Coins) e.g. Coin Change (LC 322)

SB → RBR BREAKDOWN

State: `dp[a]` = min coins to make target amount `a`.
Brainstorm Choices: Try every coin `c` where `c <= a`.
Recurrence: `dp[a] = min(dp[a], 1 + dp[a - c])`.
Base: `dp[0] = 0`. Initialize all other entries to `amount + 1`.

TEMPLATE — LC 322

```
public int coinChange(int[] coins, int amount) {
  int[] dp = new int[amount + 1];
  Arrays.fill(dp, amount + 1); // Fill with infinity sentinel!
  dp[0] = 0; // Base case: 0 amount needs 0 coins
  for (int a = 1; a ⩽ amount; a++) {
    for (int c : coins) {
      if (a - c ⩾ 0) {
        dp[a] = Math.min(dp[a], 1 + dp[a - c]);
      }
    }
  }
  return dp[amount] > amount ? -1 : dp[amount];
}
```

PATTERN 3 LONGEST INCREASING SUBSEQUENCE (Patience Sorting Binary Search)

e.g. Longest Increasing Subsequence (LC 300)

PATIENCE SORTING O(N LOG N)

Maintain array `tails` storing smallest tail element of all increasing subsequences of length `i+1`. Use binary search `lowerBound` to replace or extend tails array!

TEMPLATE — LC 300

```
public int lengthOfLIS(int[] nums) {
  int[] tails = new int[nums.length];
  int len = 0;
  for (int x : nums) {
    int i = 0, j = len;
    while (i < j) {
      int mid = i + (j - i) / 2;
      if (tails[mid] < x) i = mid + 1;
      else j = mid;
    }
    tails[i] = x;
    if (i == len) len++;
  }
  return len;
}
```

PATTERN 4 UNIQUE PATHS (2D Grid Top-Left to Bottom-Right) e.g. Unique Paths (LC 62)

SB → RBR BREAKDOWN

State: `dp[r][c]` = unique paths to cell `(r, c)`.
Brainstorm Choices: Arrived from top `(r-1, c)` OR left `(r, c-1)`.
Recurrence: `dp[r][c] = dp[r-1][c] + dp[r][c-1]`.

TEMPLATE — LC 62 (SPACE OPTIMIZED O(N))

```
public int uniquePaths(int m, int n) {
  int[] row = new int[n];
  Arrays.fill(row, 1); // Top row base case: 1 path
  for (int r = 1; r < m; r++) {
    for (int c = 1; c < n; c++) {
      row[c] += row[c - 1]; // row[c] = top + left!
    }
  }
  return row[n - 1];
}
```

DYNAMIC PROGRAMMING MASTERCLASS

Dry Run, Proofs & Mastery Cheat Sheet

🔍 DRY RUN — Coin Change ([1, 2, 5], amount = 11)

Amount (a)	Coin Try	dp[a] Recurrence (`1 + dp[a - c]`)	dp[a] Result
1	1	`1 + dp[0] = 1`	1
2	1, 2	`min(1+dp[1], 1+dp[0]) = 1`	1
5	5	`1 + dp[0] = 1`	1
11	5	`1 + dp[6] = 1 + 2 = 3` (coins 5+5+1)	3 ✅

📐 MATHEMATICAL PROOF — PRINCIPLE OF OPTIMAL SUBSTRUCTURE

Claim: An optimal solution to a DP problem contains optimal solutions to its subproblems.

Proof: If a subproblem solution were non-optimal, we could replace it with a smaller subproblem cost, strictly decreasing the total cost. This contradicts the minimality of the optimal overall solution \rightarrow **Exact Proof of Correctness!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Apply the SB → RBR → MTS framework to any DP problem
- ☐ Code House Robber in O(1) space
- ☐ Code Coin Change unbounded knapsack in O(Amount)
- ☐ Write Longest Increasing Subsequence in O(N log N)
- ☐ Code 2D String LCS in O(M · N) time
- ☐ Explain why 0/1 Knapsack loops BACKWARDS in 1D array
- ☐ Prove the Principle of Optimal Substructure
- ☐ Convert Top-Down recursion to Bottom-Up tabulation

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — State Definition is Everything

Always define your DP state (`dp[i]`) in plain English BEFORE writing any code. If your state definition is clear, the recurrence relation falls into place effortlessly!

Rule 2 — Initialize DP Table with Infinity Sentinels

For minimum cost/coin problems, initialize all `dp[]` entries with an infinity sentinel (`amount + 1` or `1e9`), and set base case `dp[0] = 0`!